GLIMSETALK

An project report Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

by

G. Jithendra Sai Chandra Baradvaj

2200030332

Under the esteemed guidance of

Mr.Sarath Chandra
HR
Gaint clout Technologies Pvt Ltd

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

K L UNIVERSITY

Green Fields, Vaddeswaram, Tadepalli,
Guntur - 522 502, Andhra Pradesh.

2025-26

Gaint clout technologies Pvt ltd,
Orbit Knowledge City, Raidurg,
Hyderabad, Telangana 500019

Bonafide Certificate

This is to certify that this project report entitled **"Glimsetalk"** submitted to **Department of Computer Science & Engineering, KL Deemed to be University, Guntur,** in connection with the University internship program is a bonafide record of work done by "**G. Jithendra Sai Chandra Baradvaj"** under my supervision at the **"Gaint clout technologies Pvt Ltd**" from "**14/07/2025**" to "**08/11/2025**".

Mr. Sarath Chandra
HR

KL Deemed to be university
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Certificate

This is to certify that the project entitled "**Glimsetalk**" is a software development and simulation work carried out by **Gali Jithendra Sai Chandra Baradvaj (2200030332)**, in partial fulfillment of the requirements for the award of the **Bachelor of Technology** degree in the **Department of Computer and Science Engineering** during the academic year **2025–2026.** The project has been approved as it satisfies the academic requirements.

Department Coordinator **Head of the Department**

University Guide **External**

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to **Mr. K. Prasad**, Chairman, and **Mr. K. Srinivasa Rao**, CEO of **Gaint Clout Technology Pvt. Ltd.** Regional Office: **Raidurg, Orbit Knowledge City, Hyderabad, Telangana – 500019**, for providing us with the opportunity to undertake our project internship at their esteemed organization. We are deeply thankful to our company guide **Mr. Sharath Chandra (HR)**, along with **Ms. Nikitha**, **Ms. Rohini**, and **Mr. Elisha**, for their constant support, guidance, and valuable insights throughout the internship period.

We extend our sincere thanks to our university guide **Dr. Ch. Venkateswara Rao**, for his encouragement and academic guidance. Our special thanks to **Dr. P. V. Chalapathy**, Director of Practice School, and **Dr. B. Suresh**, Practice School In-Charge, for providing us with this invaluable learning experience. We are also grateful to **Mrs. Sk. Thaseentaj**, Department Coordinator, for her continuous support and coordination.

Our project, titled **"Glimsetalk,"** was successfully completed as part of our Practice School program at **Gaint Clout Technology Pvt. Ltd.** The team comprised **M.Ravikishore** as the **Team Lead and Frontend developer**, **G. Jithendra Sai Chandra Baradvaj** as the **Back-End Developer**.

We sincerely appreciate the collaborative efforts, technical mentorship, and teamwork that contributed to the successful completion of this project.

Project Associates
2200032250-M.Ravikishore
2200030332 - G. Jithendra

ABSTRACT

The **Glimsetalk** project is an AI-driven conversational platform inspired by **ChatGPT**, designed to deliver human-like interactions through intelligent automation and real-time communication. The primary objective of this project is to develop a robust, secure, and feature-rich backend architecture that supports scalable chat operations, data management, and multimedia processing.

As a **Backend** Developer, my contributions focused on implementing **RESTful APIs** to handle dynamic chat operations, user authentication, and session tracking. The system stores individual user chat histories in a database and provides endpoints such as /history for retrieving previous conversations and /user for managing user profile data. Additional features include support for file uploads, PDF/Word/PPT generation (using **ReportLab**, **python-docx**, and **python-pptx**), and OCR-based text extraction from images using **OpenCV** and **Tesseract**, enabling seamless integration of document and media-based interactions.

To enhance usability and personalization, the backend supports text-to-speech (gTTS) conversion, language translation via **Googletrans/DeepL**, and emotion detection using **DeepFace/Fer**. It was about session tracking, feedback collection, and chat analytics. User actions such as logins, uploads, and feedback are timestamped and logged in the database for auditing and monitoring.

The system ensures data security through CORS, CSRF protection, token-based authentication with refresh logic, and input validation. Advanced error handling, API timeout management, and structured logging improve reliability. It was followed by comprehensive testing with **Postman**.

Overall, ***Glimsetalk*** demonstrates the integration of artificial intelligence, multimedia data handling, and secure backend design to build a production-ready, ChatGPT-like conversational ecosystem.

TABLE OF CONTENTS

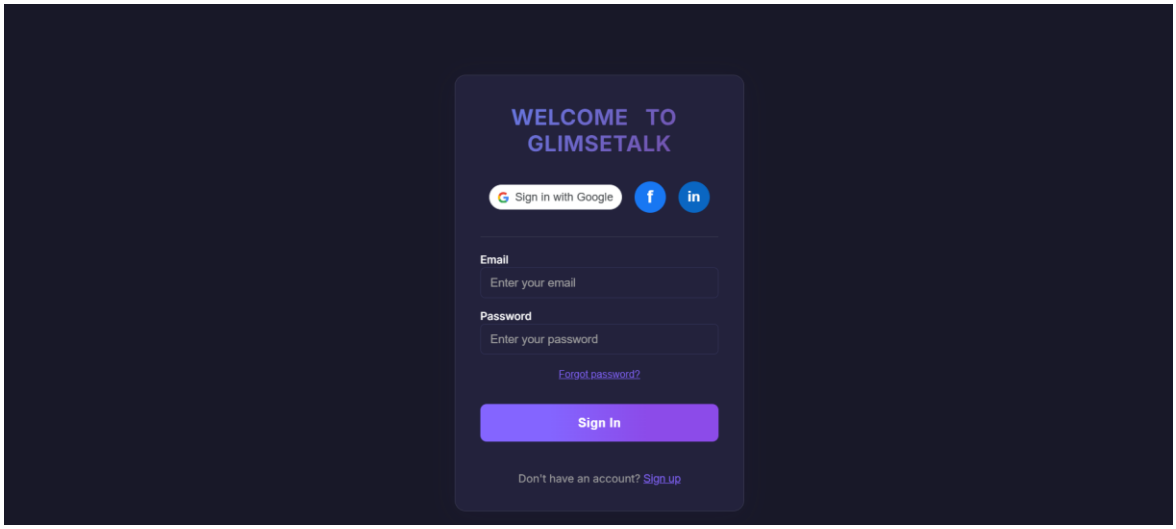
S.NO	TITLE	PAGE NO
1	Certificate	**3**
2	Acknowledgement	**4**
3	Abstract	**5**
4	Table of Contents	**6**
5	List of Figures	**7 - 10**
6	Introduction	11
7	System Analysis	**12**
8	System Design and Architecture	**13 - 14**
9	Implementation	**15 - 33**
10	Testing and Deployment	**34 - 44**
11	Results and Discussion	**45**
12	Condition and Future Scope	**46**
13	References	**47**

1. 5 List of Figures(outputs)

```
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)> cd Glimsetalk-project1
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\Glimsetalk-project1> py app.py
2025-11-02 22:53:21.764944: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-11-02 22:53:25.660986: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING:tensorflow:From C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\venv\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.spa
rse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

Frontend path exists? False
Frontend path: C:\Users\Jithendra\OneDrive\Desktop\My- Project\project-gaint\project-gaint\project-gaint\glimsetalk\dist
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
WARNING:tensorflow:From C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\venv\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.spa
rse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

Frontend path exists? False
Frontend path: C:\Users\Jithendra\OneDrive\Desktop\My- Project\project-gaint\project-gaint\project-gaint\glimsetalk\dist
 * Debugger is active!
 * Debugger PIN: 321-171-958
```

```
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\Glimsetalk-project1> cd..
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)> cd project-gaint
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\project-gaint> cd project-gaint
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\project-gaint\project-gaint> cd project-gaint
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\project-gaint\project-gaint\project-gaint> cd project-gaint
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\project-gaint\project-gaint\project-gaint\project-gaint> cd glimsetalk
(venv) PS C:\Users\Jithendra\OneDrive\Desktop\Glimsetalk Project(Gaint)\project-gaint\project-gaint\project-gaint\project-gaint\glimsetalk> npm start

> glimsetalk@0.0.0 start
> concurrently "npm run backend" "npm run dev"

(node:23876) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
[0]
[0] > glimsetalk@0.0.0 backend
[0] > cd backend && npm start
[0]
[1]
[1] > glimsetalk@0.0.0 dev
[1] > vite
[1]
[0]
[0] > glimsetalk-backend@1.0.0 start
[0] > node server.js
[0]
[1]
[1]   VITE v7.1.0  ready in 1740 ms
[1]
```

```
[1]   ➜  Local:   http://localhost:5173/
[1]   ➜  Network: use --host to expose
[0] ⚠️No SMTP configured. OTPs will be logged to console (DEV MODE).
[0] 🚀 ====================================
[0] 🚀 GlimTalk Backend Server Started
[0] 🚀 ====================================
[0] 📍 Server URL: http://localhost:5000
[0] 🔐 OTP: Enabled (TTL 10m, resend cooldown 60s)
[0] 📷 Image upload: http://localhost:5000/api/upload/image
[0] 📁 File upload: http://localhost:5000/api/upload
[0] 🔍 Debug users: http://localhost:5000/api/debug/users
[0] 🚀 ====================================
[0] ✅ Ready to accept connections!
```


WELCOME TO GLIMSETALK
Sign in with Google
Email
Enter your email
Password
Enter your password
Forgot password?
Sign In
Don't have an account? Sign up

CHAPTER 1
INTRODUCTION

The rapid advancement of artificial intelligence and natural language processing (NLP) has revolutionized human–computer interaction, leading to the development of intelligent conversational platforms such as **ChatGPT**. These systems enable users to communicate with AI models in real-time, creating opportunities for automation, information retrieval, and personalized assistance.

The **Glimsetalk** project is designed as an AI-based conversational platform inspired by **ChatGPT**, capable of processing natural language input and generating context-aware responses. The platform integrates various backend services to manage user sessions, chat history, multimedia inputs, and data analytics efficiently. Its architecture ensures seamless interaction between users and the AI model, offering features such as chat storage, file processing, language translation, and emotion detection.

The backend system of **Glimsetalk** is developed using **Flask**, a Python-based web framework known for its flexibility and scalability. It includes **RESTful APIs** that handle user authentication, message processing, and file operations. Additional capabilities such as audio generation (using gTTS), text extraction from images (via OpenCV and Tesseract), and document creation in multiple formats (PDF, Word, and PPT) enhance the platform's functionality.

Overall, **Glimsetalk** represents a robust, production-ready AI communication platform that demonstrates the integration of intelligent chat systems, multimedia handling, and cloud-based backend design. It showcases the power of combining artificial intelligence with scalable and secure backend technologies to create efficient, user-friendly conversational experiences.

CHAPTER 2
SYSTEM ANALYSIS

1. System Architecture

The backend architecture of **Glimsetalk** follows a **modular, service-oriented structure** developed using the **Flask** framework. The system consists of key layers:

Presentation Layer: The frontend (web or mobile client) that interacts with the backend through RESTful APIs.

Application Layer: The Flask-based server responsible for handling API requests, authentication, data processing, and integration with AI models and third-party services.

Database Layer: A structured database (such as MySQL or PostgreSQL) used to store user profiles, chat history, uploaded files, and feedback records.

AI & Utility Layer: Handles AI model communication, language translation (via Googletrans/DeepL), emotion detection (via DeepFace/Fer), and file processing (OpenCV, Tesseract, python-docx, python-pptx, reportlab).

2. Functional Modules

The system analytics identifies several functional modules:

User Management Module: Handles registration, login, logout, token management, and user profile retrieval through secure API endpoints.

Chat Management Module: Manages user sessions, stores chat histories, and provides /history routes for past conversation retrieval.

File Processing Module: Supports uploading files (PDF, Word, PPT, ZIP, Images), extracts text using appropriate libraries, and logs uploads in the database.

Audio & Translation Module: Converts chat responses into speech (gTTS), translates conversations, and adapts replies based on user preferences.

Payment & Feedback Module: Integrates payment gateways (Stripe/Razorpay), handles webhooks, and records user feedback for continuous improvement.

3. Data Flow

The user sends a request (chat message or file) to the backend through the API.

The request passes through validation and authentication layers.

The backend processes input (text, file, or image), invokes the AI model or corresponding service, and generates a response.

The response is stored in the database along with metadata like timestamp and session ID.

The result is returned to the frontend in real-time, optionally converted to audio or translated.

4. Performance & Security Analytics

Caching: Implemented using Redis to reduce response latency for frequently accessed data.

Monitoring: Integrated Prometheus and Grafana for API performance visualization and error rate tracking.

Security: Enforced CORS, CSRF, and token-based authentication to prevent unauthorized access.

Error Handling: Global error handler ensures consistent and informative error responses for API failures.

5. Outcome

The system analytics demonstrate that *Glimsetalk* efficiently integrates AI-based conversation, secure data handling, and real-time communication features within a scalable backend environment. It ensures optimized performance, high availability, and strong data consistency across modules.

CHAPTER 3
SYSTEM DESIGN AND ARCHITECTURE

3.1 Technology Stack

Programming Language: Python

Framework: Flask

Database: MySQL / PostgreSQL

Caching: Redis

AI & NLP: OpenAI API, Googletrans, DeepFace

Text-to-Speech: gTTS

Image Processing: OpenCV, Tesseract OCR

Document Handling: ReportLab, python-docx, python-pptx

Testing Tool: Postman

Security: CORS, CSRF Tokens, JWT Authentication

· 3.2 Schemas

User Schema Structure

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
username	varchar(50)	NO	UNI	NULL	
email	varchar(150)	NO	UNI	NULL	
phone	varchar(15)	YES	UNI	NULL	
date_of_birth	date	YES		NULL	
address	varchar(250)	YES		NULL	
password_hash	varchar(255)	NO		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
audio_enabled	tinyint(1)	YES		1	

Chat Schema Structure

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
session_id	varchar(36)	NO		NULL	
prompt	text	NO		NULL	
response	text	NO		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

File Upload Log Schema Structure

```
59 •  DESC file_upload_log;
60 •  SELECT * FROM chat;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
filename	varchar(255)	NO		NULL	
upload_time	datetime	YES		NULL	

Support Feedback Structure

```
59 •  DESC support_feedback;
60 •  SELECT * FROM chat;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	YES	MUL	NULL	
name	varchar(100)	NO		NULL	
email	varchar(120)	NO		NULL	
subject	varchar(255)	NO		NULL	
message	text	NO		NULL	
created_at	datetime	YES		NULL	

CHAPTER 4

IMPLEMENTATION

1. (.env) file

```
glimsetalk-project1 > .env
APP_SECRET_KEY=mysecretkey123
JWT_SECRET_KEY=myjwtsecret123
DATABASE_URL=mysql+pymysql://root:Glimsetalk@127.0.0.1/glimsetalk

GOOGLE_OAUTH_CLIENT_ID=851031352219-j7ce71c26tksnl2h9989g1g1fa2j582a.apps.googleusercontent.com
GOOGLE_OAUTH_CLIENT_SECRET=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

FACEBOOK_OAUTH_CLIENT_ID=9125504350907893
FACEBOOK_OAUTH_CLIENT_SECRET=ed1a1b590fc820fbe4cd191d98f9bed0

LINKEDIN_OAUTH_CLIENT_ID=86mjlrtwayhwmb
LINKEDIN_OAUTH_CLIENT_SECRET=WPL_AP1.kTFEtalT3HGszTy8.PUc42A==

GEMINI_API_KEY=AIzaSyAgxKz_2woAbnBteDuhIwkA58r9nURrP3M
```

2. Social Logins Auth

```
google_bp = make_google_blueprint(
    client_id=os.getenv("GOOGLE_OAUTH_CLIENT_ID"),
    client_secret=os.getenv("GOOGLE_OAUTH_CLIENT_SECRET"),
    scope=[
        "openid",
        "https://www.googleapis.com/auth/userinfo.email",
        "https://www.googleapis.com/auth/userinfo.profile"
    ],
    redirect_to="google_final"
)
app.register_blueprint(google_bp, url_prefix="/login")

facebook_bp = make_facebook_blueprint(
    client_id=os.getenv("FACEBOOK_OAUTH_CLIENT_ID"),
    client_secret=os.getenv("FACEBOOK_OAUTH_CLIENT_SECRET"),
    scope=["email"],
    redirect_to="facebook_final"
)
app.register_blueprint(facebook_bp, url_prefix="/login")

linkedin_bp = make_linkedin_blueprint(
    client_id=os.getenv("LINKEDIN_OAUTH_CLIENT_ID"),
    client_secret=os.getenv("LINKEDIN_OAUTH_CLIENT_SECRET"),
    scope=["openid", "profile", "email"],
    redirect_to="linkedin_final"
)
app.register_blueprint(linkedin_bp, url_prefix="/login")
```

3. Register Route

```python
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json() or {}

    username = data.get('username')
    email = data.get('email')
    password_input = data.get('password') or data.get('password_input')
    phone = data.get('phone')

    date_of_birth_str = data.get('date_of_birth') or data.get('dob_obj')
    address = data.get('address')

    if not username or not email or not password_input:
        return jsonify({"message": "Username, email, and password are required!"}), 400
    if User.query.filter_by(email=email).first():
        return jsonify({"message": "Email already registered!"}), 400
    if phone and User.query.filter_by(phone=phone).first():
        return jsonify({"message": "Phone number already registered!"}), 400

    dob_obj = None
    if date_of_birth_str:
        try:
            dob_obj = datetime.strptime(date_of_birth_str, "%Y-%m-%d").date()
        except ValueError:
            return jsonify({"message": "Invalid date format! Use YYYY-MM-DD"}), 400

    user = User(
        username=username,
        email=email,
        password_hash=generate_password_hash(password_input),
        phone=phone,
        date_of_birth=dob_obj,
        address=address
    )

    db.session.add(user)
    db.session.commit()
```

4. Login

```python
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json() or {}

    email = data.get("email")
    password_input = data.get("password")

    if not email or not password_input:
        return jsonify({"message": "Email and password are required!"}), 400

    user = User.query.filter_by(email=email).first()

    if not user or not check_password_hash(user.password_hash, password_input):
        return jsonify({"message": "Invalid credentials!"}), 401
    token = create_access_token(identity=str(user.id))
    refresh_token = create_refresh_token(identity=str(user.id))

    return jsonify({
        "access_token": token,
        "refresh_token": refresh_token,
        "user": user.to_dict()
    }), 200
```

5. Social Logins routing confirm

```python
@app.route("/login/google/final")
def google_final():
    if not google.authorized:
        return redirect(url_for("google.login"))
    resp = google.get("/oauth2/v2/userinfo")
    if not resp.ok:
        return "Failed to fetch Google user info", 400
    data = resp.json()

    handle_social_login(data.get('email'), data.get('name') or 'Google User')
    return redirect("https://www.google.com")

@app.route("/login/facebook/final")
def facebook_final():
    if not facebook.authorized:
        return redirect(url_for("facebook.login"))
    resp = facebook.get("/me?fields=id,name,email")
    if not resp.ok:
        return "Failed to fetch Facebook user info", 400
    data = resp.json()
    return handle_social_login(data.get("email"), data.get("name"))

@app.route("/linkedin/final")
def linkedin_final():
    if not linkedin.authorized:
        return redirect(url_for("linkedin.login"))

    resp = linkedin.get("/userinfo")
    if not resp.ok:
        return "Failed to fetch profile info from LinkedIn!", 400

    userinfo = resp.json()
    name = userinfo.get("name") or f"{userinfo.get('given_name', '')} {userinfo.get('family_name', '')}".strip()
    email = userinfo.get("email")

    if not email:
        return jsonify({"error": "LinkedIn did not return an email"}), 400

    return handle_social_login(email, name)
```

6. Chat Stream

```python
@app.route("/chat/stream", methods=["POST"])
@jwt_required()
def chat_stream():
    current_user_id = get_jwt_identity()
    data = request.get_json() or {}
    prompt = data.get("prompt")

    if not prompt:
        return jsonify({"error": "Prompt is required"}), 400

    session_id = data.get("session_id") or str(uuid.uuid4())

    try:
        model = genai.GenerativeModel("gemini-1.5-flash")
        result = model.generate_content(prompt)
        response_text = result.text if result and result.text else "No response from Gemini."

    except Exception as e:
        err_str = str(e).lower()
        if "timeout" in err_str:

            return jsonify({"error": "Gemini API timed out. Please try again later."}), 504
        elif "rate limit" in err_str:
            return jsonify({"error": "Rate limit exceeded. Please wait and retry."}), 429
        else:
            return jsonify({"error": f"Error calling Gemini: {str(e)}"}), 502

    try:
        chat_entry = Chat(user_id=current_user_id, session_id=session_id, prompt=prompt, response=response_text)
        db.session.add(chat_entry)
        db.session.commit()
    except Exception as db_error:
        db.session.rollback()
        return jsonify({"error": f"Database error: {str(db_error)}"}), 500

    def generate():
        for word in response_text.split():
            yield word + " "
            time.sleep(0.3)
```

7. Chats (Get) and Login Activity

```
@app.route("/chats", methods=["GET"])
@jwt_required()
def get_chats():
    current_user_id = get_jwt_identity()
    chats = Chat.query.filter_by(user_id=current_user_id).all()
    return jsonify([c.to_dict() for c in chats])
                                (function) methods: Any
@app.route("/login-activities", methods=["GET"])
@jwt_required()
def get_login_activities():
    current_user_id = get_jwt_identity()
    activities = LoginActivity.query.filter_by(user_id=current_user_id).all()
    return jsonify([a.to_dict() for a in activities])
```

8. Download Audio

```
@app.route('/download_audio/<filename>', methods=['GET'])
@jwt_required()
def download_audio(filename):
    filepath = os.path.join("uploads", filename)
    if not os.path.exists(filepath):
        return jsonify({"error": "File not found"}), 404

    # Determine audio mimetype (supports .mp3 and .wav here, extend as needed)
    ext = filename.lower().split('.')[-1]
    if ext == "mp3":
        mimetype = "audio/mpeg"
    elif ext == "wav":
        mimetype = "audio/wav"
    else:
        mimetype = "application/octet-stream"

    return send_file(
        filepath,
        mimetype=mimetype,
        as_attachment=True,  # Prompts download
        download_name=filename  # For Flask >=2.0
    )
```

9. Voice to text

```
@app.route("/voice_to_text", methods=["POST"])
@jwt_required()
def voice_to_text():
    if "file" not in request.files:
        return jsonify({"error": "No audio file uploaded"}), 400

    file = request.files["file"]
    if file.filename == "":
        return jsonify({"error": "Empty filename"}), 400
```

```
    filename = secure_filename(file.filename)
    filepath = os.path.join("uploads", filename)
    os.makedirs("uploads", exist_ok=True)
    file.save(filepath)
```

```
    if filepath.lower().endswith(".mp3"):
        try:
            from pydub import AudioSegment
            sound = AudioSegment.from_mp3(filepath)
            wav_path = filepath.rsplit(".", 1) + ".wav"
            sound.export(wav_path, format="wav")
            filepath = wav_path
        except Exception as conv_err:
            return jsonify({"error": f"MP3 conversion failed: {str(conv_err)}"}), 500
```

```
    recognizer = sr.Recognizer()
    try:
        with sr.AudioFile(filepath) as source:
            audio = recognizer.record(source)
            text = recognizer.recognize_google(audio)
    except sr.UnknownValueError:
        return jsonify({"error": "Could not understand the audio"}), 400
    except sr.RequestError as api_err:
        return jsonify({"error": f"Speech recognition API error: {str(api_err)}"}), 500
    except Exception as e:
        return jsonify({"error": f"Speech recognition failed: {str(e)}"}), 500
```

```
    try:
        current_user_id = get_jwt_identity()
        voice_entry = VoiceToText(
            user_id=current_user_id,
            filename=filename,
            transcript=text
        )
        db.session.add(voice_entry)
        db.session.commit()
    except Exception as db_err:
        db.session.rollback()
        return jsonify({"error": f"Database error: {str(db_err)}"}), 500
```

```
    # Include download URL of audio
    download_url = url_for('download_audio', filename=filename, _external=True)
```

```
    return jsonify({
        "filename": filename,
        "transcript": text,
        "download_url": download_url,
        "message": "Transcript saved successfully!",
        "record": voice_entry.to_dict()
```

```
}), 201
```

10. History

```
@app.route("/history", methods=["GET"])
@jwt_required()
def history():
    current_user_id = get_jwt_identity()

    limit = request.args.get("limit", type=int)

    query = Chat.query.filter_by(user_id=current_user_id).order_by(Chat.created_at.desc())
    if limit:
        query = query.limit(limit)

    chats = query.all()
    return jsonify([c.to_dict() for c in chats])
```

11. History to pdf conversion

```
@app.route("/history/pdf", methods=["GET"])
@jwt_required()
def history_pdf():
    current_user_id = get_jwt_identity()
    chats = Chat.query.filter_by(user_id=current_user_id).order_by(Chat.created_at.asc()).all()

    if not chats:
        return jsonify({"error": "No chat history found"}), 404
```

```
    # Create in-memory file
    buffer = io.BytesIO()
    p = canvas.Canvas(buffer, pagesize=letter)
    width, height = letter
```

```
    # Title
    p.setFont("Helvetica-Bold", 16)
    p.drawString(200, height - 50, "Chat History")
```

```
    # Start text
    y = height - 100
    p.setFont("Helvetica", 10)
```

```
    for chat in chats:
        text = f"User: {chat.prompt}"
        p.drawString(50, y, text)
        y -= 15
```

```
        text = f"Bot: {chat.response}"
        p.drawString(70, y, text)
        y -= 30
```

```
        if y < 50:  # Start new page if space runs out
            p.showPage()
            y = height - 50
            p.setFont("Helvetica", 10)
```

```
    p.save()
    buffer.seek(0)
```

```
    return send_file(
```

```
    buffer,
    as_attachment=True,
    download_name="chat_history.pdf",
    mimetype="application/pdf"
)
```

12. Generate to word

```
@app.route('/generate_word', methods=['GET'])
def generate_word():
    doc = Document()
    doc.add_heading('Document Title', 0)
    doc.add_paragraph('This is a paragraph in Word.')
    buffer = io.BytesIO()
    doc.save(buffer)
    buffer.seek(0)
    return send_file(
        buffer,
        as_attachment=True,
        download_name="generated.docx",
        mimetype="application/vnd.openxmlformats-officedocument.wordprocessingml.document"
    )
```

13. Generate PPT

```
@app.route('/generate_ppt', methods=['GET'])
def generate_ppt():
    prs = Presentation()
    slide_layout = prs.slide_layouts[0]
    slide = prs.slides.add_slide(slide_layout)
    title = slide.shapes.title
    subtitle = slide.placeholders[1]
    title.text = "Hello, World!"
    subtitle.text = "python-pptx was here!"
    buffer = io.BytesIO()
    prs.save(buffer)
    buffer.seek(0)
    return send_file(
        buffer,
        as_attachment=True,
        download_name="generated.pptx",
        mimetype="application/vnd.openxmlformats-officedocument.presentationml.presentation"
    )
```

14. Upload_Zip file

```
@app.route('/upload_zip', methods=['POST'])
@jwt_required()
def upload_zip():
    if 'file' not in request.files:
        return jsonify({'error': 'No file part in the request'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400

    filename = secure_filename(file.filename)
    upload_folder = 'uploads'
    os.makedirs(upload_folder, exist_ok=True)
    filepath = os.path.join(upload_folder, filename)
    file.save(filepath)

    try:
        current_user_id = get_jwt_identity()
        upload_log = FileUploadLog(user_id=current_user_id, filename=filename)
        db.session.add(upload_log)
        db.session.commit()
    except Exception as e:
        db.session.rollback()
        return jsonify({'error': f'Failed to log upload: {str(e)}'}), 500

    return jsonify({'message': f'File {filename} uploaded and logged successfully.'}), 200
```

15. Ppt to text

```
@app.route('/upload_ppt_extract_text', methods=['POST'])
@jwt_required()
def upload_ppt_extract_text():
    if 'file' not in request.files:
        return jsonify({"error": "No file part in request"}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({"error": "No selected file"}), 400

    if not file.filename.lower().endswith('.pptx'):
        return jsonify({"error": "Only .pptx files are allowed"}), 400

    filename = secure_filename(file.filename)
    upload_folder = 'uploads'
    os.makedirs(upload_folder, exist_ok=True)
    filepath = os.path.join(upload_folder, filename)
    file.save(filepath)

    try:
        prs = Presentation(filepath)
        extracted_texts = []
        for slide in prs.slides:
            for shape in slide.shapes:
                if shape.has_text_frame:
                    text = shape.text.strip()
                    if text:
                        extracted_texts.append(text)

        full_text = "\n".join(extracted_texts)
    except Exception as e:
        return jsonify({"error": f"Failed to process PPTX: {str(e)}"}), 500

    return jsonify({
        "message": f"File '{filename}' uploaded and text extracted successfully",
        "extracted_text": full_text
    }), 200
```

16. Chat with file

```
@app.route('/chat_with_file', methods=['POST'])
@jwt_required()
def chat_with_file():
    try:
        current_user_id = get_jwt_identity()

        # Get user prompt
        prompt = None
        if request.is_json:
            data = request.get_json()
            prompt = data.get('prompt')
        else:
            prompt = request.form.get('prompt')
```

```
        if not prompt:
            return jsonify({"error": "Prompt is required"}), 400
```

```
        # Handle file upload (optional)
        extracted_text = ""
        if 'file' in request.files:
            file = request.files['file']
            # Example: process PPTX file text extraction if .pptx
            if file.filename.lower().endswith('.pptx'):
                from pptx import Presentation
                import io
```

```
                file_stream = io.BytesIO(file.read())
                prs = Presentation(file_stream)
                texts = []
                for slide in prs.slides:
                    for shape in slide.shapes:
                        if shape.has_text_frame:
                            texts.append(shape.text.strip())
                extracted_text = "\n".join(texts)
```

```
        # Add other file type handlers here (images, ZIP, etc.)
```

```
    # Combine prompt with extracted file text for chatbot input
    combined_input = f"{prompt}\n\nContext from file:\n{extracted_text}"
```

```
    # Call your existing Gemini model or chatbot logic
    try:
        model = genai.GenerativeModel("gemini-1.5-flash")
        result = model.generate_content(combined_input)
        response_text = result.text if result and result.text else "No response from Gemini."
    except Exception as e:
        response_text = f"Error calling Gemini: {str(e)}"
```

```
    # Save chat history in DB
    try:
        chat_entry = Chat(user_id=current_user_id, prompt=combined_input, response=response_text)
        db.session.add(chat_entry)
        db.session.commit()
    except Exception as db_error:
        db.session.rollback()
        return jsonify({"error": f"Database error: {str(db_error)}"}), 500
```

```
    return jsonify({
        "extracted_text": extracted_text,
        "chat_reply": response_text,
        "chat_record": chat_entry.to_dict()
    }), 201
```

```
  except Exception as e:
    return jsonify({"error": f"Chat with file failed: {str(e)}"}), 500
```

17. Detect Text

```
@app.route('/detect_text', methods=['POST'])
@jwt_required()
def detect_text():
    if 'image' not in request.files:
        return jsonify({'error': 'No image uploaded'}), 400

    file = request.files['image']
    in_bytes = file.read()
    npimg = np.frombuffer(in_bytes, np.uint8)
    image = cv2.imdecode(npimg, cv2.IMREAD_COLOR)
    if image is None:
        return jsonify({'error': 'Invalid image file'}), 400

    newW, newH = (320, 320)
    image_resized = cv2.resize(image, (newW, newH))
    blob = cv2.dnn.blobFromImage(image_resized, 1.0, (newW, newH),
                                 (123.68, 116.78, 103.94), swapRB=True, crop=False)
    net = cv2.dnn.readNet('frozen_east_text_detection.pb')
    net.setInput(blob)
    scores, _ = net.forward([
        "feature_fusion/Conv_7/Sigmoid",
        "feature_fusion/concat_3"
    ])

    thresh = 0.5
    contains_text = bool((scores[0, 0] > thresh).any())

    return jsonify({
        "contains_text": contains_text
    }), 200
```

18. Detect emotion

```
@app.route('/detect_emotion', methods=['POST'])
def detect_emotion():
    if 'image' not in request.files:
        return jsonify({'error': 'No image uploaded'}), 400

    file = request.files['image']
    image_bytes = file.read()

    # Decode image bytes into OpenCV format
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    try:
        # Analyze emotion
        result = DeepFace.analyze(img, actions=['emotion'], enforce_detection=False)

        # Handle single/multiple faces
        if isinstance(result, list):
            dominant_emotion = result[0].get('dominant_emotion', 'unknown')
        else:
            dominant_emotion = result.get('dominant_emotion', 'unknown')

        return jsonify({'emotion': dominant_emotion}), 200

    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

19. Logout and Upload logs

```
@app.route('/upload_logs', methods=['GET'])
@jwt_required()
def upload_logs():
    current_user_id = get_jwt_identity()
    logs = FileUploadLog.query.filter_by(user_id=current_user_id).order_by(FileUploadLog.upload_time.desc()).all()
    return jsonify([log.to_dict() for log in logs])

@app.route('/logout', methods=['POST'])
@jwt_required()
def logout():
    jti = get_jwt()["jti"]
    blacklist.add(jti)
    return jsonify({"message": "Successfully logged out"}), 200
```

20. Support

```
@app.route("/support", methods=["POST"])
@jwt_required(optional=True)
def support_feedback_route():
    data = request.get_json() or {}

    name = data.get("name")
    email = data.get("email")
    subject = data.get("subject")
    message = data.get("message")

    if not name or not email or not subject or not message:
        return jsonify({"error": "All fields are required"}), 400

    current_user_id = get_jwt_identity()

    try:
        feedback = SupportFeedback(
            user_id=current_user_id,
            name=name,
            email=email,
            subject=subject,
            message=message
        )
        db.session.add(feedback)
        db.session.commit()
        return jsonify({
            "message": "Feedback submitted successfully!",
            "feedback": feedback.to_dict()
        }), 201
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": f"MySQL error: {str(e)}"}), 500
```

21. User audio

```python
@app.route("/user/audio", methods=["POST"])
@jwt_required()
def set_audio_preference():
    current_user_id = get_jwt_identity()
    data = request.get_json() or {}
    audio_enabled = bool(data.get("audio_enabled", True))
    user = User.query.get(current_user_id)
    if not user:
        return jsonify({"message": "User not found"}), 404
    user.audio_enabled = audio_enabled
    db.session.commit()
    return jsonify({"audio_enabled": user.audio_enabled}), 200

@app.route("/user/audio", methods=["GET"])
@jwt_required()
def get_audio_preference():
    current_user_id = get_jwt_identity()
    user = User.query.get(current_user_id)
    if not user:
        return jsonify({"message": "User not found"}), 404
    return jsonify({"audio_enabled": user.audio_enabled}), 200
```

```python
@app.route('/process', methods=['POST'])
def process_file():
    # Example endpoint for processing files
    file = request.files['file']
    # Perform processing (e.g., using frozen_east_text_detection.pb)
    result = {"message": "File processed successfully"}
    return jsonify(result)

@app.route("/api/test")
def test():
    return {"message": "API is live!"}

@app.route('/')
def serve_frontend():
    return send_from_directory(app.static_folder, 'index.html')

# Handle other frontend routes (React Router etc.)
@app.errorhandler(404)
def not_found(e):
    return send_from_directory(app.static_folder, 'index.html')


if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    app.run(debug=True,port=5000)
```

22. From user

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	username	email	phone	date_of_birth	address	password_hash
1	jithendra	jithendra@example.com	9876543210	2005-08-23	Hyderabad	pbkdf2:sha256:1000000$tlsjicCeaREQKru3$2b...
2	Santosh	santosh@gmail.com	9846359127	2005-08-27	Hyderabad	pbkdf2:sha256:1000000$JDIhGKuToH7k4W7N$...
3	Kluinternprojects	kluinternprojects@gmail.com	NULL	NULL	NULL	pbkdf2:sha256:1000000$mQIEfnVp0CI8eUzi$d...
4	Jithendra Sai Chandra Bardvaj	jittu220723@gmail.com	NULL	NULL	NULL	pbkdf2:sha256:1000000$Y6wdUwJiIztmHkcY$a...
5	Nandan	nandan@gmail.com	9856231475	2005-08-30	Vijayawada	pbkdf2:sha256:1000000$aym9cEYhip7MJnoF$b...
9	Rohit	rohit@gmail.com	9063810659	2005-01-11	Ongole	scrypt:32768:8:1$pZGpiu9t30Ma1gNB$172178...
10	meghana	meghana@gmail.com	9959225689	2004-10-10	Ongole	scrypt:32768:8:1$RStfJZOpMxvuDBz0$164ead...
11	Vijay	vijay@gmail.com	9562351847	2004-10-01	Hyderabad	scrypt:32768:8:1$xjpEMCnl4rQChXvJ$4b64d1b...
14	JohnDoe	johndoe@example.com	7584518814	2000-05-10	123 Street, City	scrypt:32768:8:1$6Z5cyMK4QXKBaMc7$52ea2...
NULL	NULL	NULL	NULL	NULL	NULL	NULL

23. From chat

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	user_id	session_id	prompt	response	created_at
1	2		Hello AI!	You said: Hello AI!. (Simulated response)	2025-08-26 18:28:46
2	5		Hello AI, how are you?	[MOCK RESPONSE] You said: 'Hello AI, how are ...	2025-08-28 07:23:04
3	5		Tell me a fun fact about space.	[MOCK RESPONSE] You said: 'Tell me a fun fact ...	2025-08-28 07:25:17
4	5		Explain the difference between machine learnin...	[MOCK RESPONSE] You said: 'Explain the differ...	2025-08-28 07:26:18
5	5		Hello chatbot, how are you?	Hello chatbot, how are you?!	2025-08-29 17:45:24
6	5		Hello chatbot, how are you?		2025-08-29 17:51:01
7	5		Hello chatbot, how are you?	... :	2025-08-29 17:55:03
8	5		Hello chatbot, how are you?	bot	2025-08-29 18:00:24
9	5		Hello chatbot, how are you?	[Local LLaMA-2 error: HTTPConnectionPool(host...	2025-08-29 18:07:20
10	5		Can you tell me a joke?	[Local LLaMA-2 error: HTTPConnectionPool(host...	2025-08-29 18:07:53
11	5		Can you tell me a joke?	Sorry, I couldn't generate a reply. Try again.	2025-08-29 18:24:50
12	5		Hello chatbot, how are you?	Sorry, I couldn't generate a reply. Try again.	2025-08-29 18:25:07
13	5		Tell me a joke	Sorry, I couldn't generate a reply. Try again.	2025-08-29 19:42:14
14	5		Tell me a joke	[Local Mistral error: HTTPConnectionPool(host='...	2025-08-29 20:10:43
15	5		Tell me a joke	[Local Mistral error: HTTPConnectionPool(host='...	2025-08-29 20:23:59
16	5		Tell me a joke	I'm having some technical difficulties, but here's...	2025-08-29 20:25:05
17	5		Explain the difference between machine learnin...	Machine Learning is a broader field that involve...	2025-08-29 20:28:09
18	5		hello how are you	[Local Mistral error: HTTPConnectionPool(host='...	2025-08-30 09:18:24

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	user_id	session_id	prompt	response	created_at
19	5		hello how are you	[Local Mistral error: 500 Server Error: Internal S...	2025-08-30 09:36:25
20	5		Explain the difference between machine learnin...	[Local Mistral error: 500 Server Error: Internal S...	2025-08-30 09:37:36
21	5		Explain the difference between machine learnin...	[Local Mistral error: 500 Server Error: Internal S...	2025-08-30 09:38:35
22	5		Explain the difference between machine learnin...	[Local Mistral error: 500 Server Error: Internal S...	2025-08-30 09:44:29
23	5		Hello AI	[Local Mistral error: HTTPConnectionPool(host='...	2025-09-01 12:03:55
24	5		Hello bot, how are you?	You said: Hello bot, how are you?	2025-09-05 17:18:28
25	5		Write a funny joke about AI and humans.	Why was the AI so bad at hide-and-seek? Beca...	2025-09-05 17:27:07
26	5		Generate 5 creative startup ideas in the educati...	1. **Personalized Learning Adventure Game Pla...	2025-09-05 17:27:56
27	5		Voice file: WhatsApp_Audio_2025-08-30_at_14...	hello how are you	2025-09-09 11:08:55
28	5		Generate 5 creative startup ideas in the educati...	1. **Personalized Learning Journeys Powered b...	2025-09-09 18:46:05
29	5		Hello, how are you?	I'm doing well, thank you for asking! How are y...	2025-09-09 18:49:18
30	9		Tell me a joke	Why don't scientists trust atoms? Because the...	2025-09-09 19:53:06
31	5		Tell me a joke	Why don't scientists trust atoms? Because the...	2025-09-10 11:58:20
32	10		what is the difference between maggie and Ra...	While both Maggie and Ramen are noodle soups...	2025-09-11 23:44:16
33	5		Tell me about this presentation Context from file:	Please provide me with the file! I need the cont...	2025-09-22 13:43:39
34	5		Tell me about this presentation Context from file:	Please provide me with the file content. I need ...	2025-09-23 11:35:20
35	5	01fcd553...	Hello, simulate streaming!	Okay, let's simulate a stream! I'll act as the str...	2025-09-24 11:29:24
36	5	79c3b07...	Write a poem about the sea.	The sea, a restless, sapphire dream, Where su...	2025-09-25 09:05:33

24. From Login Activity

Result Grid | Filter Rows: | Edit: | Export/Import: | W

id	user_id	login_time	login_method	ip_address	user_agent
1	5	2025-08-28 07:06:12	local	127.0.0.1	NULL
2	5	2025-08-29 17:21:13	local	127.0.0.1	NULL
3	5	2025-08-29 17:27:03	local	127.0.0.1	NULL
4	5	2025-08-29 19:39:44	local	127.0.0.1	NULL
5	5	2025-08-29 20:21:38	local	127.0.0.1	NULL
6	5	2025-08-30 09:05:51	local	127.0.0.1	NULL
7	5	2025-08-30 09:37:07	local	127.0.0.1	NULL
8	5	2025-09-01 12:01:35	local	127.0.0.1	NULL
9	5	2025-09-05 17:17:44	local	127.0.0.1	NULL
10	5	2025-09-05 17:54:41	local	127.0.0.1	NULL
11	5	2025-09-07 16:08:18	local	127.0.0.1	NULL
12	5	2025-09-09 07:45:30	local	127.0.0.1	NULL
13	5	2025-09-09 11:03:34	local	127.0.0.1	NULL
14	5	2025-09-09 16:45:31	local	127.0.0.1	NULL
15	5	2025-09-09 18:45:10	local	127.0.0.1	NULL
16	5	2025-09-09 18:45:20	local	127.0.0.1	NULL
NULL	NULL	NULL	NULL	NULL	NULL

25. **From file upload**

Result Grid | Filter Rows: | Edit:

id	user_id	filename	upload_time
1	5	sample_files.zip	2025-09-23 12:23:37
NULL	NULL	NULL	NULL

26. From Voice to text

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

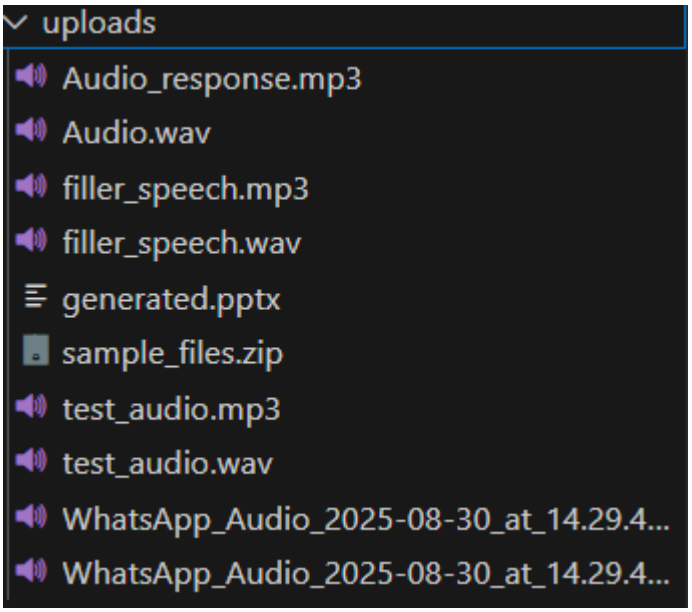
id	user_id	filename	transcript	created_at
1	5	WhatsApp_Audio_2025-08-30_at_14.29.44_02...	hello how are you	2025-09-09 16:46:03
2	5	WhatsApp_Audio_2025-08-30_at_14.29.44_02...	hello how are you	2025-09-09 16:58:57
3	5	Audio.wav	hello how are you	2025-09-09 17:01:54
4	9	Audio.wav	hello how are you	2025-09-09 20:17:09
5	9	Audio.wav	hello how are you	2025-09-10 11:15:27
6	9	Audio.wav	hello how are you	2025-09-10 11:16:44
7	10	Audio.wav	hello how are you	2025-09-11 23:47:38
NULL	NULL	NULL	NULL	NULL

27. From Support feeedback

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	user_id	name	email	subject	message	created_at
1	5	Nandan	nandan@gmail.com.com	Payment issue	I was charged twice for my subscription, please...	2025-09-24 11:02:42
NULL	NULL	NULL	NULL	NULL	NULL	NULL

28. Uploads



29. Database Query

```sql
-- Create database if not exists
CREATE DATABASE IF NOT EXISTS glimsetalk;

USE glimsetalk;

-- Create users table with extra fields
CREATE TABLE user (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,
    phone VARCHAR(15) UNIQUE,
    date_of_birth DATE,
    address VARCHAR(250),
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert sample users (passwords should be hashed in your app)
INSERT INTO user (username, email, phone, date_of_birth, address, password_hash)
VALUES
('Alice', 'alice@example.com', '1234567890', '1990-01-01', '123 Main Street', 'hashed_password1'),
('Bob', 'bob@example.com', '0987654321', '1992-05-15', '456 Elm Street', 'hashed_password2');
```

```sql
USE glimsetalk;

-- Create login_activity table
CREATE TABLE IF NOT EXISTS login_activity (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    login_method ENUM('local', 'google', 'facebook', 'linkedin') NOT NULL,
    ip_address VARCHAR(45),   -- IPv4 or IPv6
    user_agent VARCHAR(255),  -- browser/device info
    FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
);

-- Create chat table
CREATE TABLE IF NOT EXISTS chat (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    prompt TEXT NOT NULL,
    response TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
);
```

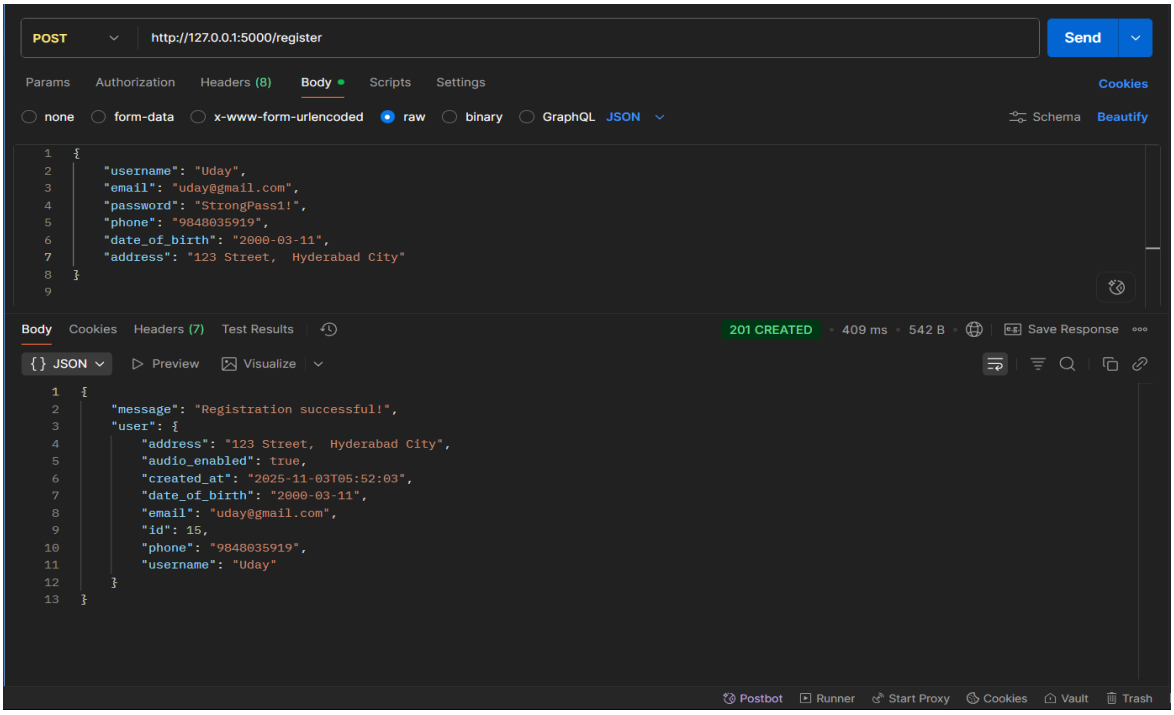
```sql
CREATE TABLE IF NOT EXISTS voice_to_text (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    filename VARCHAR(255) NOT NULL,
    transcript TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
);

CREATE TABLE file_upload_logs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  filename VARCHAR(255) NOT NULL,
  upload_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES user(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE support_feedback (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    name VARCHAR(100) NOT NULL,
```

```sql
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL,
    subject VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE SET NULL
);

ALTER TABLE chat
ADD COLUMN session_id VARCHAR(36) NOT NULL AFTER user_id;

USE glimsetalk;
DESC voice_to_text;
SELECT * FROM support_feedback;


ALTER TABLE user ADD COLUMN audio_enabled BOOLEAN DEFAULT TRUE;
```

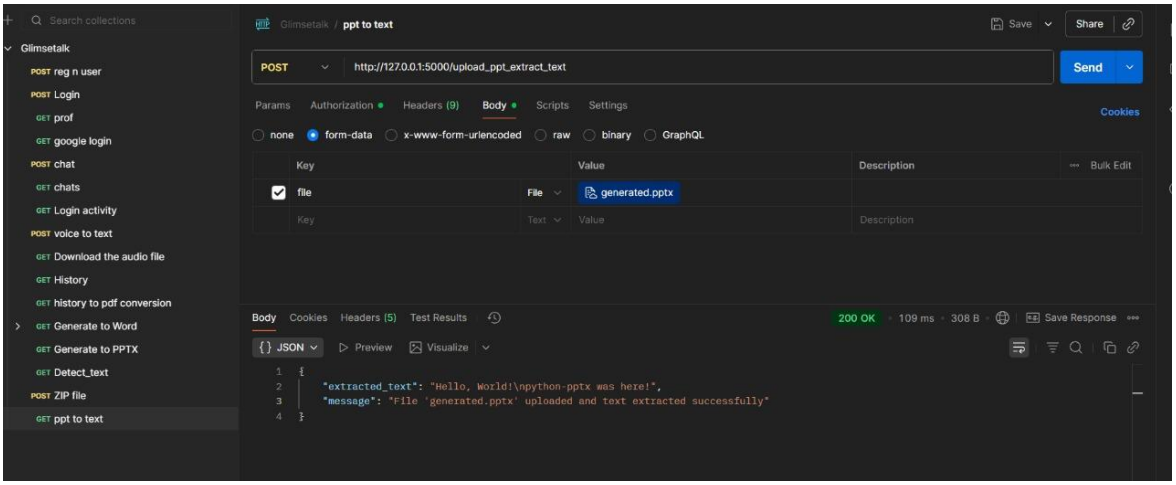
CHAPTER 5

TESTING AND DEPLOYMENT

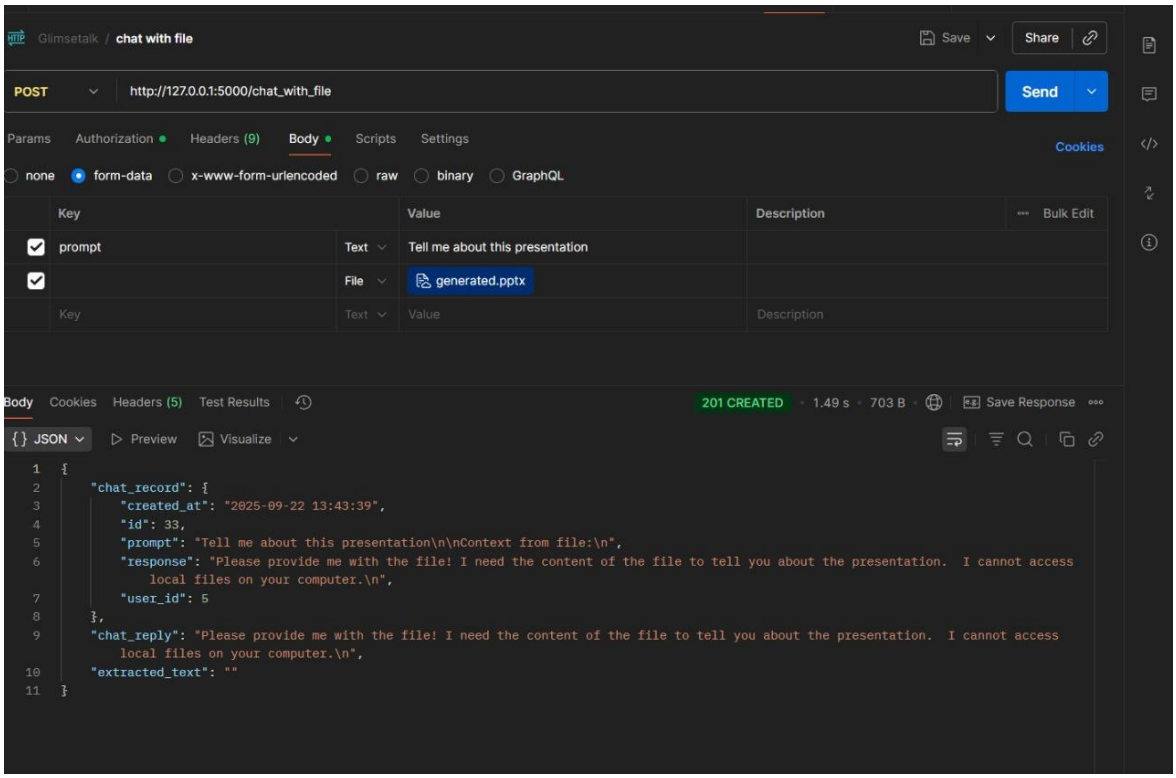
1. Register User



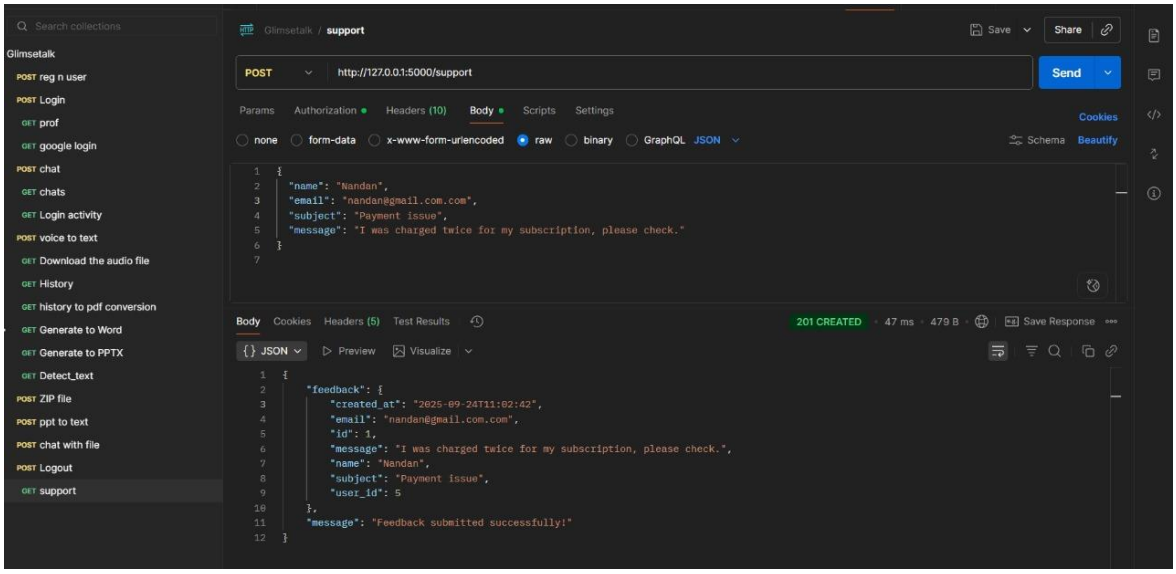
2. Ppt to text extraction



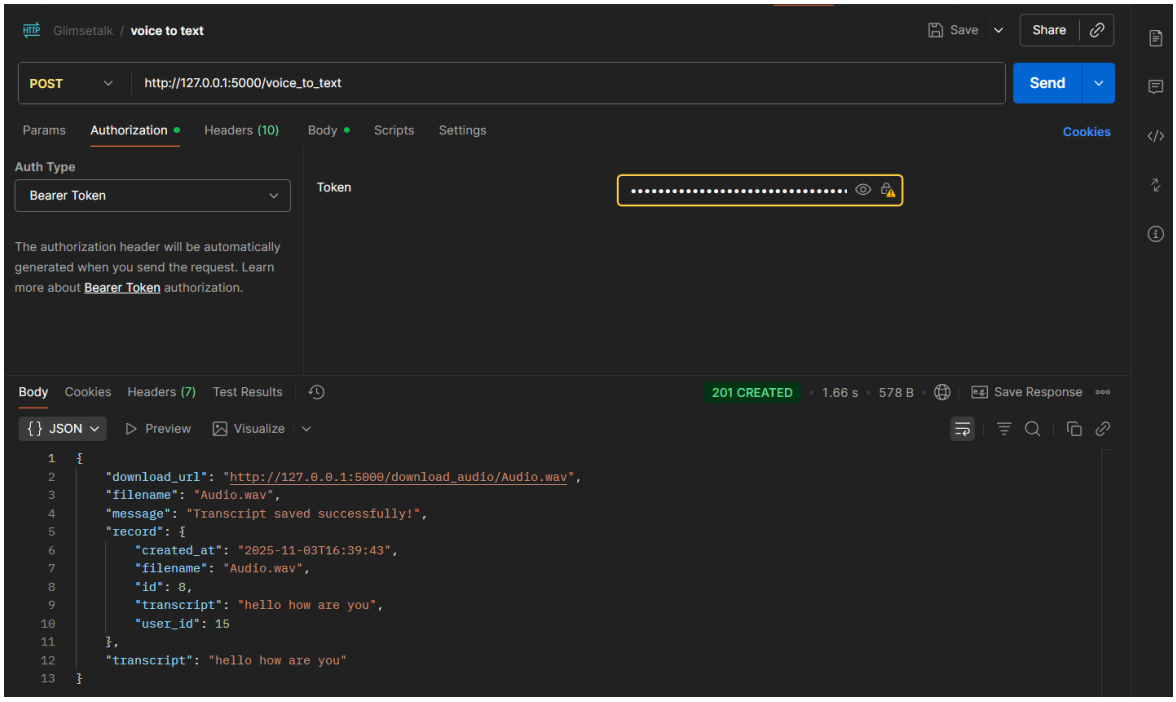
3. Chat with file



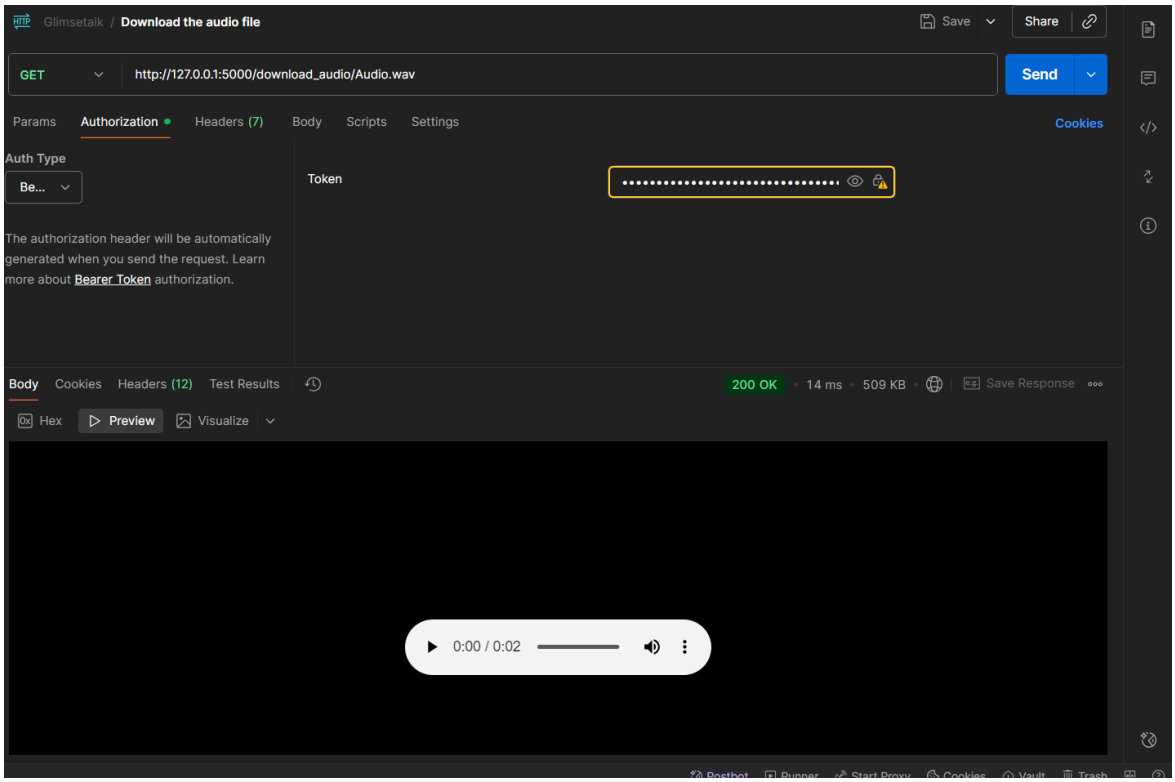
4. Support Feedback



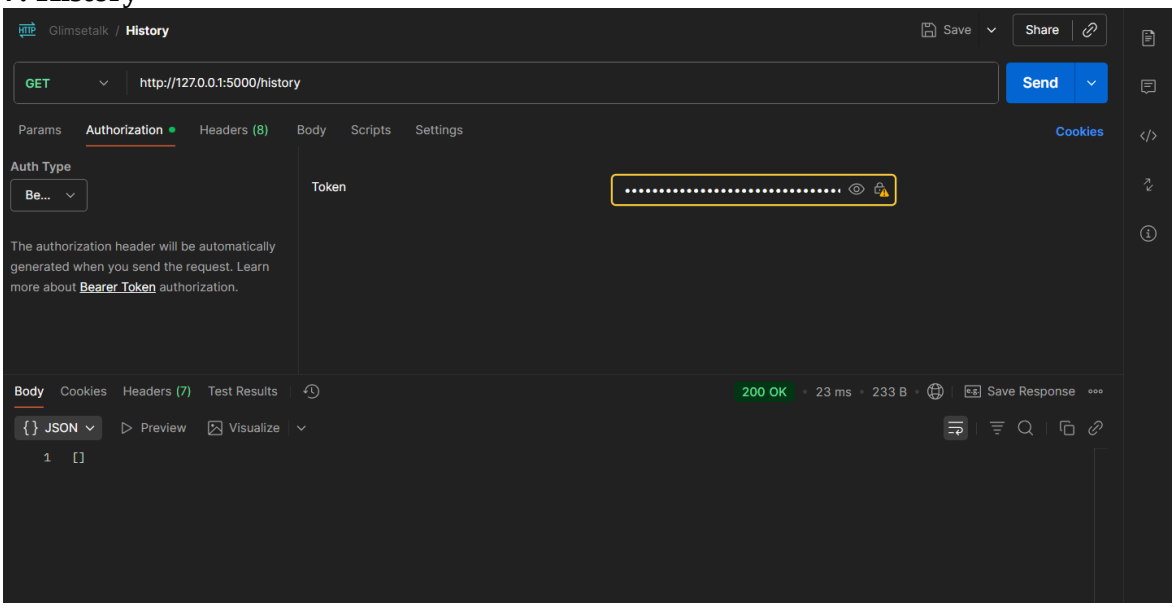
5. Voice to text



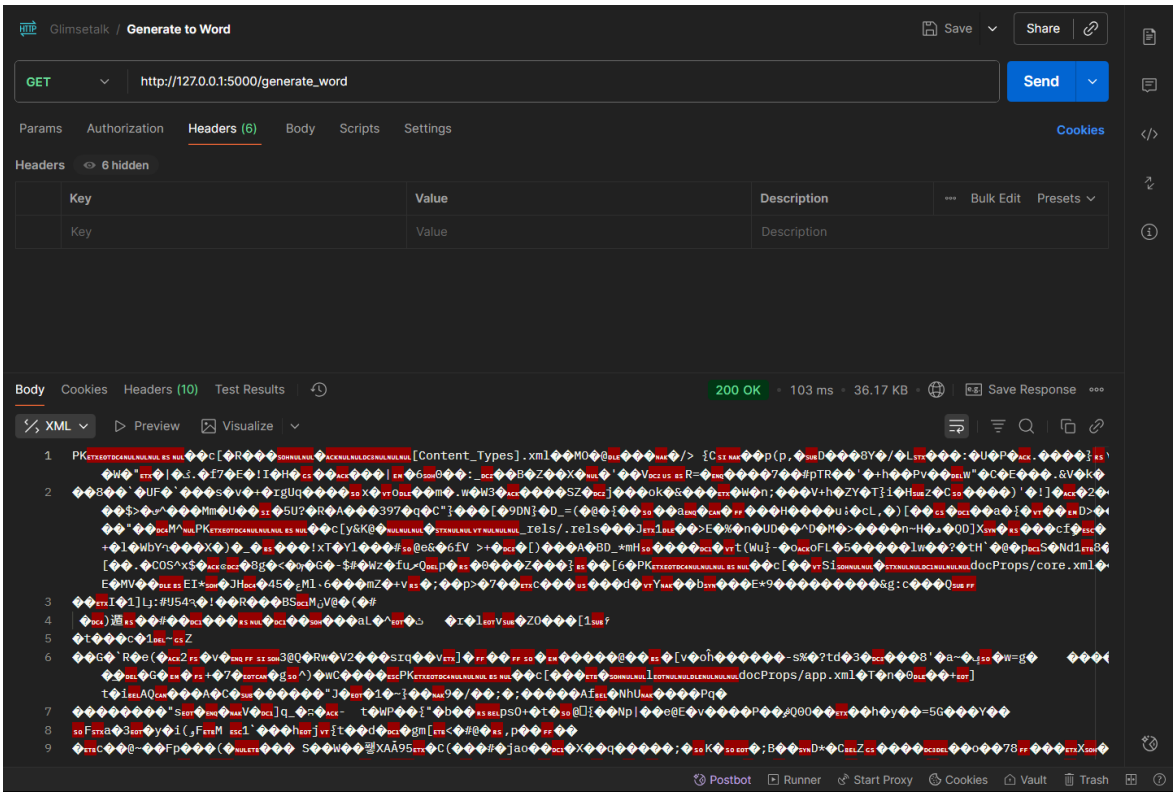
6. Download Audio File



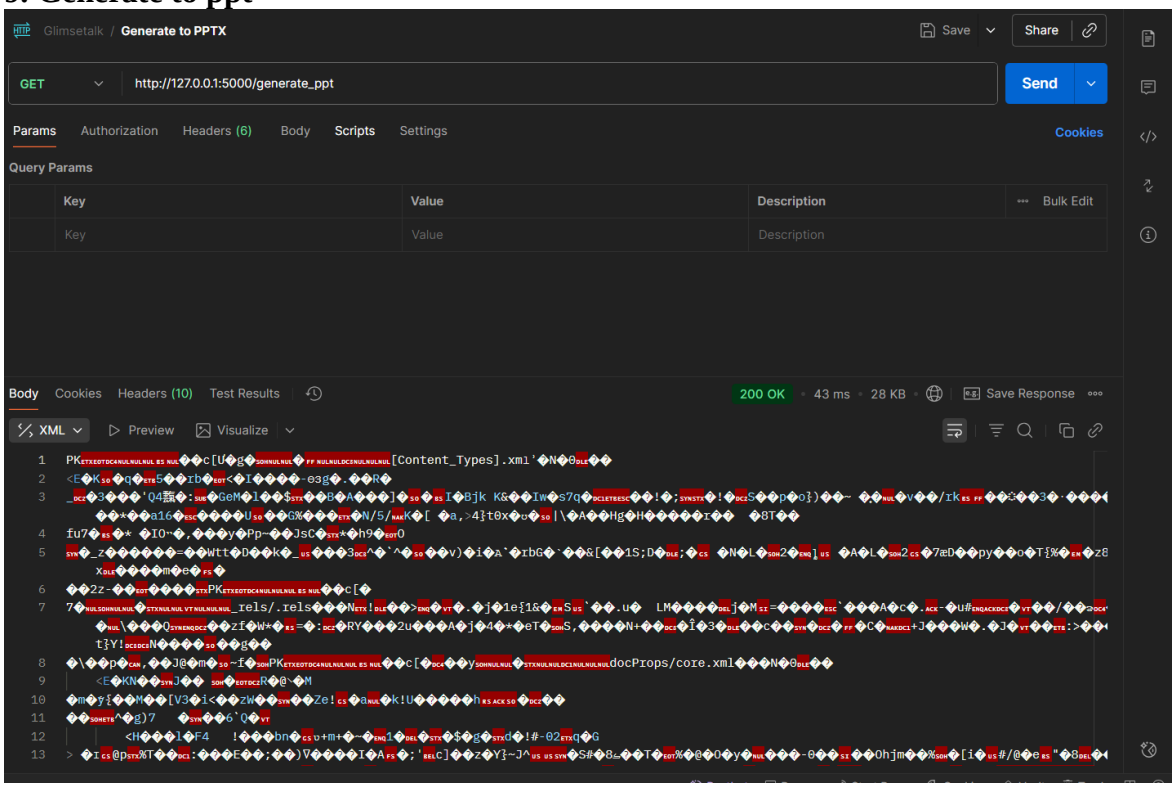
7. History



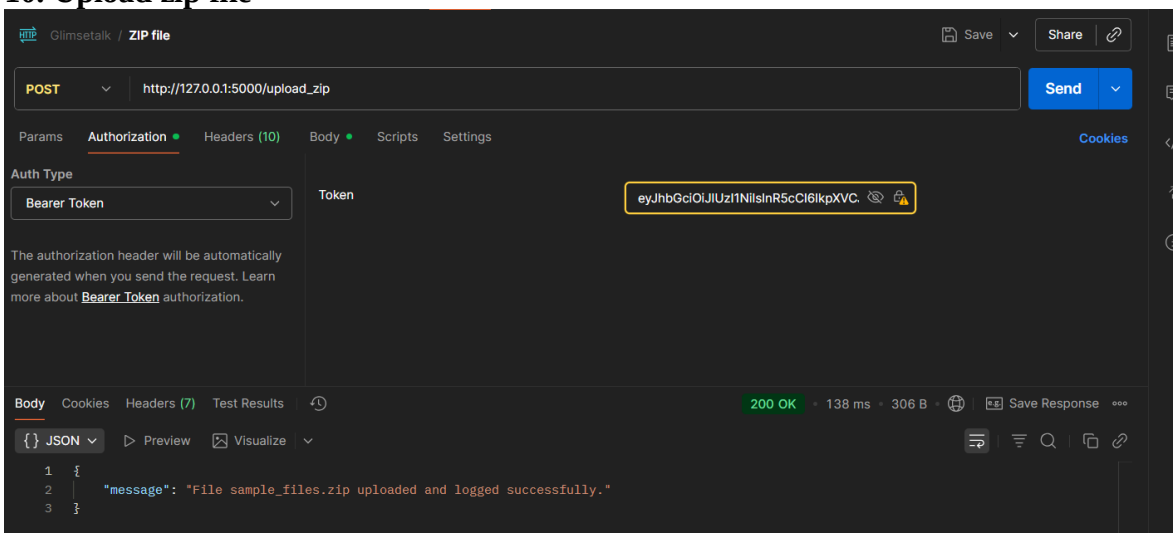
8. Generate to word



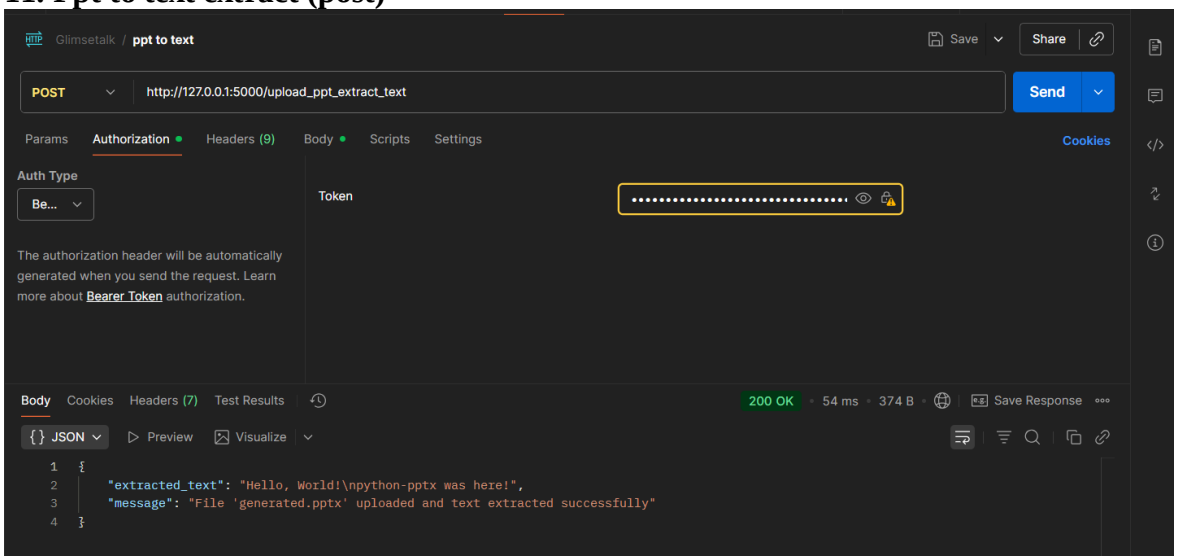
9. Generate to ppt



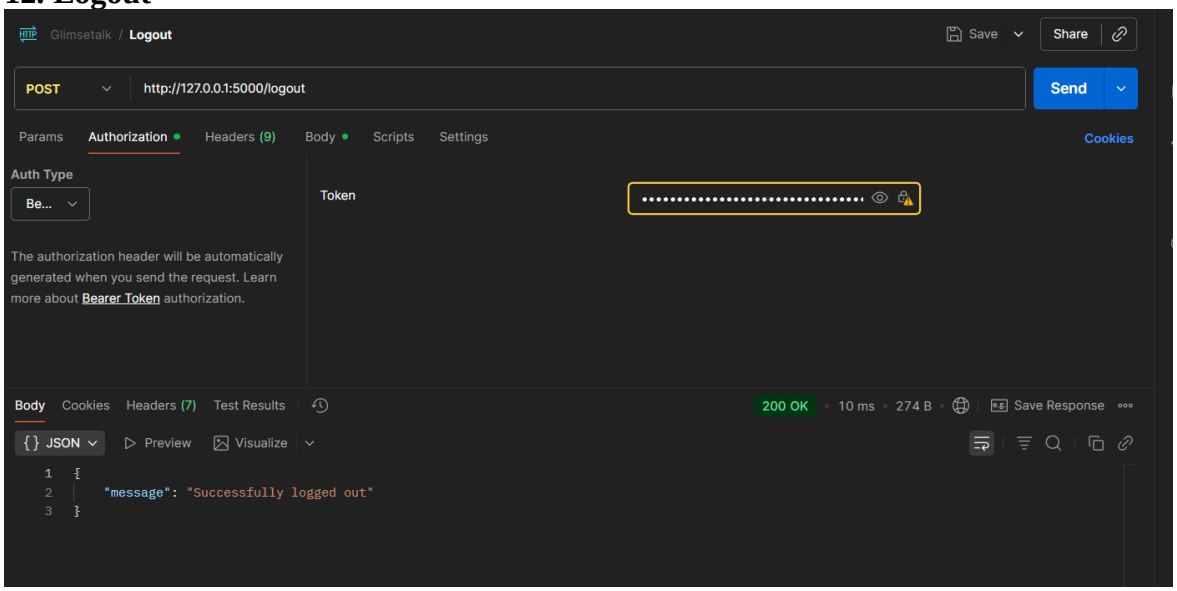
10. Upload zip file



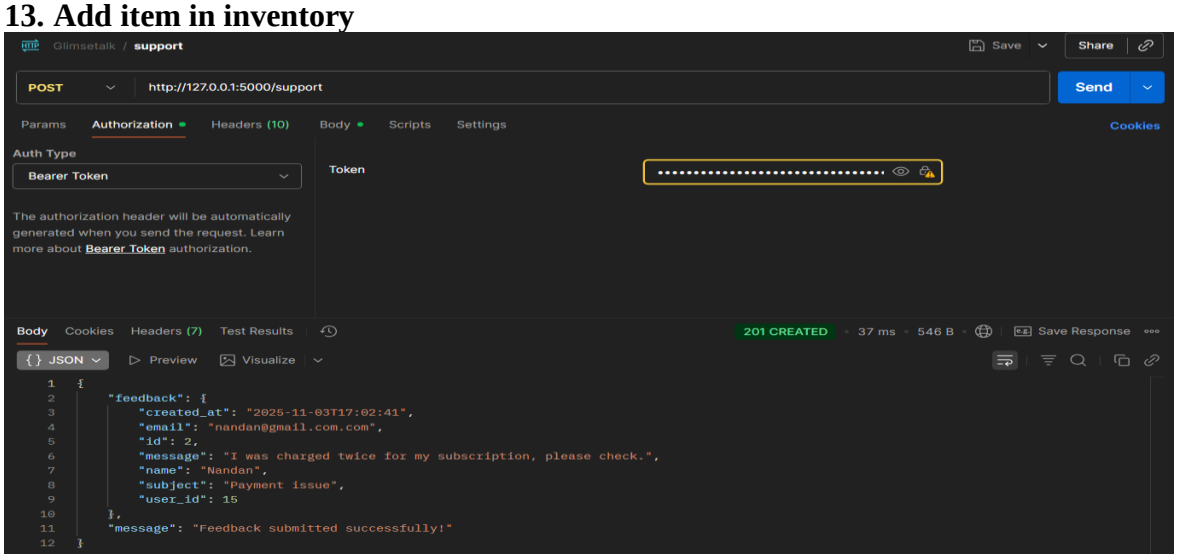
11. Ppt to text extract (post)



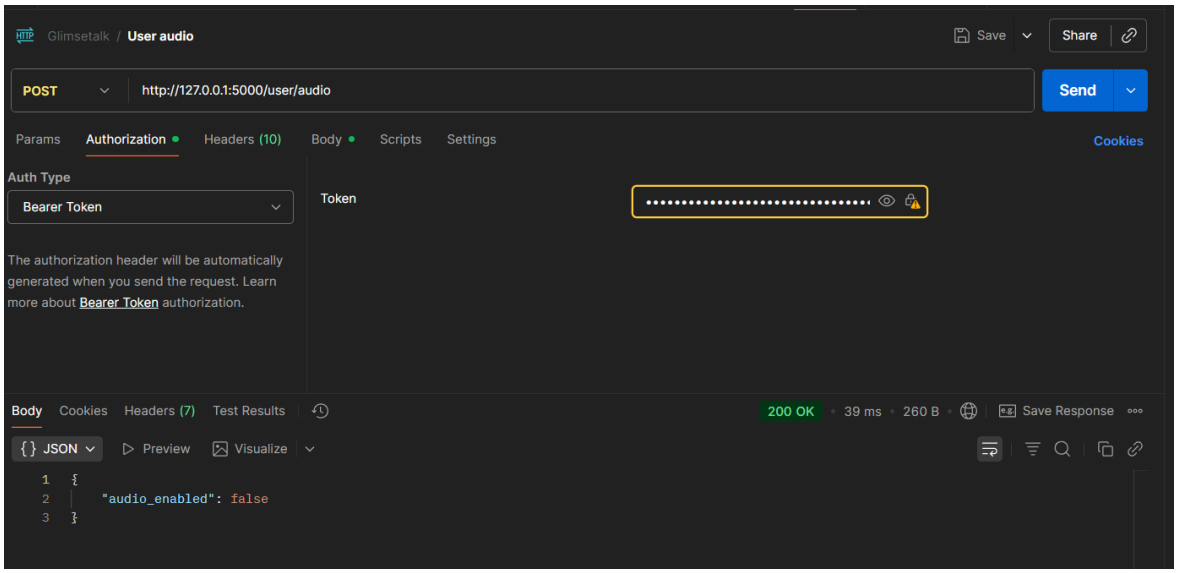
12. Logout



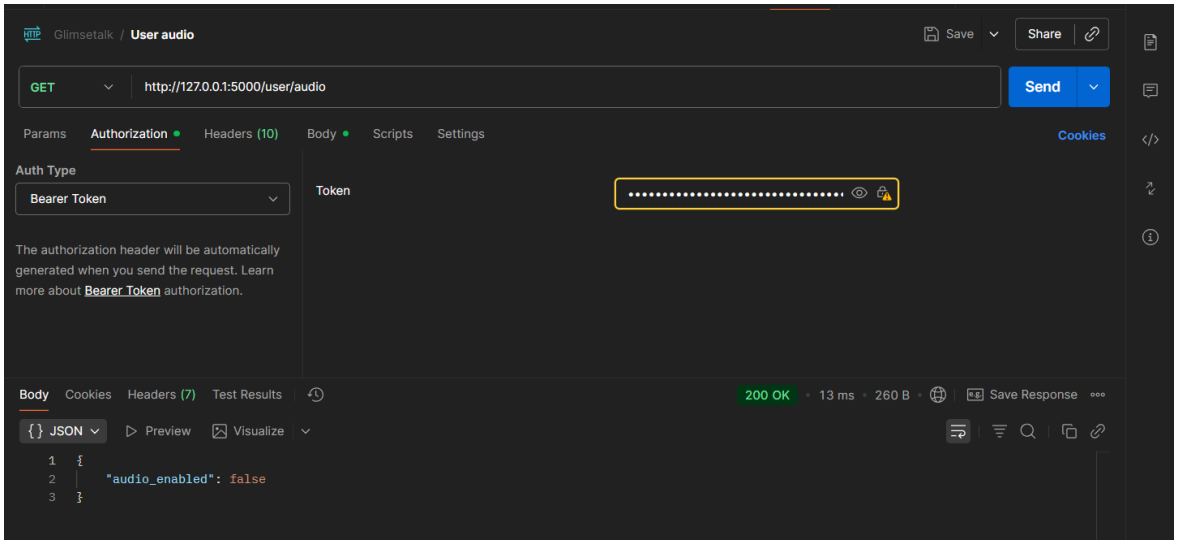
13. Add item in inventory



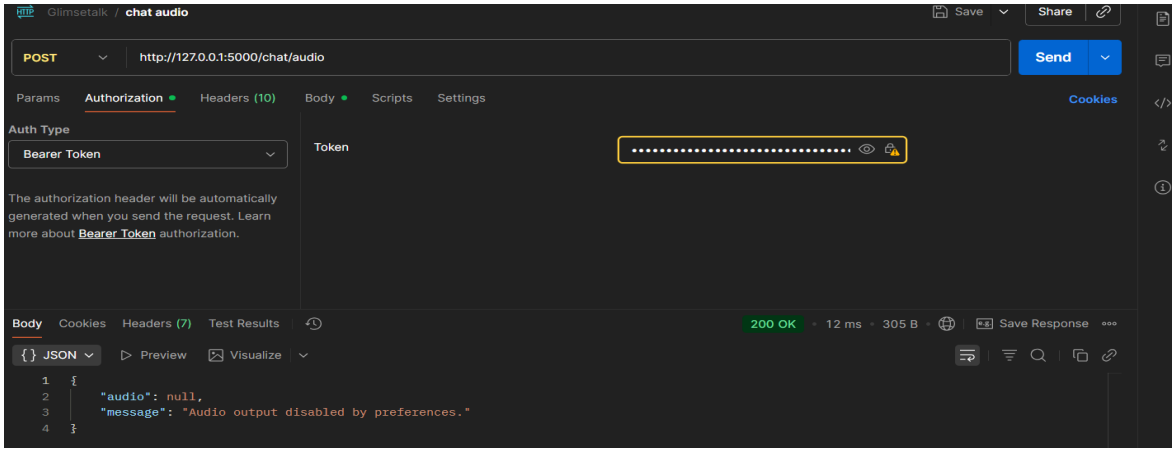
14. User Audio



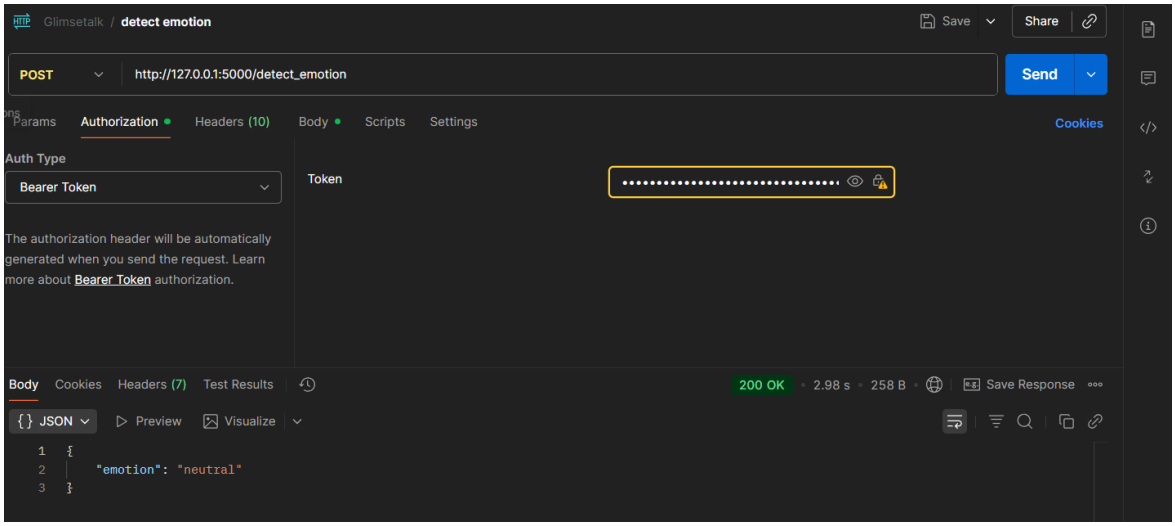
15. User Audio (Get)



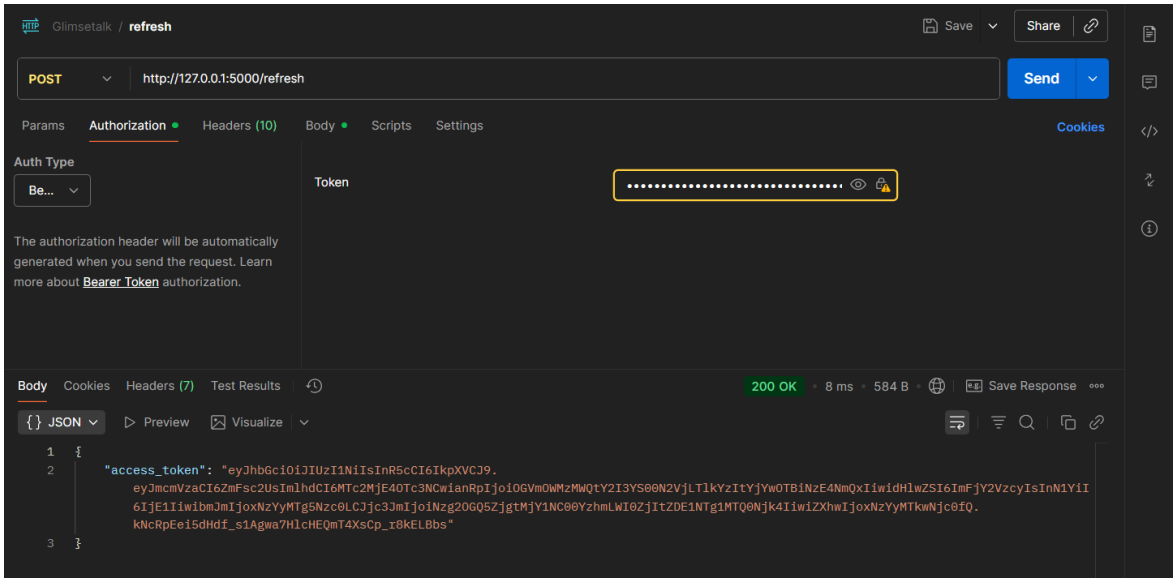
16. Chat Audio



17. Detect Emotion



18. Refresh to accesss



CHAPTER 6
RESULTS AND DISCUSSION

The **Glimsetalk** system successfully integrates advanced AI and NLP capabilities to deliver intelligent, real-time chatbot interactions. The backend efficiently handles user sessions, stores chat history, and supports multimedia processing such as image text extraction, document generation, and voice synthesis.

The system demonstrated reliable **performance and scalability**, with optimized database queries ensuring smooth retrieval of chat history and user data. The integration of **OpenAI API** improved the chatbot's contextual understanding, while **DeepFace and FER** enabled emotion-based responses for a more personalized experience.

The addition of **PDF, Word, and PPT generation** features enhanced the system's versatility, allowing users to export chat sessions easily. **Security measures** such as JWT authentication, CSRF protection, and input validation ensured data safety and integrity.

Overall, the project achieved its goal of creating a **secure, AI-powered conversational platform** with multi-modal capabilities, demonstrating robustness, flexibility, and user-centered design.

Chapter 7
CONCLUSION AND FUTURE WORK

Conclusion

The **Glimsetalk** project demonstrates the successful implementation of an advanced AI-driven **chatbot** platform that integrates multiple intelligent features such as real-time chat interaction, document generation, image text extraction, and emotion detection. The system ensures smooth and secure communication between the user and the backend, leveraging powerful technologies like **Flask**, **OpenAI API**, **DeepFace**, and **Tesseract**. With efficient database management, secure authentication mechanisms, and user-specific chat history, the platform provides a reliable and personalized user experience. Additionally, multilingual support through translation APIs enhances accessibility for users across different languages, while audio generation through text-to-speech modules makes interactions more engaging and inclusive.

In the future, **Glimsetalk** can be further enhanced by incorporating more advanced deep learning models to improve contextual understanding and emotion recognition accuracy. Integration of real-time voice-based conversation using speech-to-text and natural voice synthesis could provide a more interactive and human-like communication experience. Moreover, expanding the system into mobile and desktop applications will increase user reach and convenience. Enhancements such as cloud-based backups, real-time analytics dashboards, and microservices-based architecture can further improve scalability, performance, and fault tolerance. These developments will transform **Glimsetalk** into a more intelligent, adaptive, and versatile conversational AI platform suited for real-world applications in education, business, and customer support.

CHAPTER 8
REFERENCES

REFERENCES

· Flask Documentation – *Flask Web Framework for Python* – https://flask.palletsprojects.com

· OpenAI API Documentation – *ChatGPT and Generative AI Integration* – https://platform.openai.com/docs

· Python-docx Documentation – *Creating and Editing Word Documents in Python* – https://python-docx.readthedocs.io

· python-pptx Documentation – *Working with PowerPoint Files in Python* – https://python-pptx.readthedocs.io

· ReportLab User Guide – *PDF Generation in Python* – https://www.reportlab.com/documentation

· OpenCV Documentation – *Computer Vision and Image Processing Library* – https://docs.opencv.org

· Tesseract OCR Documentation – *Optical Character Recognition Tool* – https://tesseract-ocr.github.io

· Googletrans API – *Python Library for Google Translate* – https://py-googletrans.readthedocs.io

· DeepFace Documentation – *Facial Attribute Analysis Framework for Python* – https://github.com/serengil/deepface